

Restaurant OS

Backend + DB Phase-Wise Implementation Plan

July 2026

Contents

Phase 0 — Foundation: DB Schema, Auth, Base API Structure

Phase 1 — Super Admin Portal Backend

Phase 2 — Admin Portal Backend

Phase 3 — Warehouse Portal Backend

Phase 4 — Cloud Kitchen Portal Backend

Phase 5 — Branch Portal Backend

Phase 6 — Shared Request Engine, Notifications & Audit Log

Phase 7 — Analysis, Forecasting & Planning Backend

Phase 8 — Billing & Subscription Backend

Phase 9 — Additional Recommended Backend Work

Phase 10 — Testing, Integration & Deployment

Restaurant OS — Backend + DB Phase-Wise Implementation Plan

Stack: FastAPI (Python) + PostgreSQL + Redis · Source: Restaurant Operating System Portal Spec

Phases are ordered so that each portal's backend is buildable against a stable foundation, with the shared engines (Request workflow, Notifications, Audit log) called out as their own phase rather than duplicated per portal.

Phase 0 — Foundation: DB Schema, Auth, Base API Structure

Multi-tenancy note: every table from Phase 0 onward carries a `restaurant_id` (tenant) column and is scoped by it — Super Admin is the only role that ever queries across tenants.

Module / API area	What it covers	Key backend / DB considerations
Core DB schema	Restaurant (tenant), User, Branch, Kitchen, Warehouse, Product	Get the unified User table right first — every portal's staff live in it, and every later phase references it
Multi-tenancy scaffolding	<code>restaurant_id</code> on all tenant-scoped tables, row-level scoping helper	Cheap now, expensive to retrofit — build the scoping helper once, reuse in every route/service
Auth & RBAC	Login (single endpoint for all 5 roles), JWT + refresh tokens, role guard dependency	Role model must map exactly to “who can see whom”: a manager sees only users they created; Admin sees their whole restaurant; Super Admin sees everything
Hierarchy/visibility engine	<code>created_by_id</code> self-reference on User, scoped-query helper	This is the rule stated across every portal in the spec — implement once centrally, never let a route build its own filter
Base API structure & DB connection layer	Routing conventions, error handling, shared CRUD utilities, SQLAlchemy session management	Establish response/error shape now — inconsistent formats slow down every later phase
Dev/staging environment	Environment setup, seed data (sample restaurant, admin, branch) for local dev	Needed before Phase 1 work can be demoed end to end

Phase 1 — Super Admin Portal Backend

Module / API area	What it covers	Key backend / DB considerations
Add Restaurant API	Create restaurant + owner contact (number/email), initial branch count	Also provisions the first Admin user and triggers credential email — don't create the restaurant without its Admin in the same transaction
Plan management API	Activate/halt a restaurant's plan	Halting must be checked at the auth layer, not just hidden in the UI — a halted restaurant's users should be blocked at login or first request
Edit Admin / plan update API	Update plan tier, branch limit, Admin contact details	Changes here should write an audit log entry — this affects billing downstream
Billing data API	Next billing date, plan amount, invoice history	Feeds the Admin-facing “share bill with admin” view — same table, read-scoped by role
Share bill with Admin	Expose billing/invoice data to the Admin portal (read-only)	Simple scoped read over Billing/Invoice tables from Phase 0's schema
Credential provisioning	Generate email/password for new Admin, email it	Reuses the same credential-email utility every portal's user-creation flow uses (Phase 0) — don't build a one-off for Super Admin

Phase 2 — Admin Portal Backend

Module / API area	What it covers	Key backend / DB considerations
Add Branch / Kitchen / Warehouse API	Create locations under the Admin's restaurant	Enforce restaurant's branch_limit from Phase 1's plan before allowing creation
Add User API	Create Warehouse/Kitchen/Branch Manager accounts with credentials	Admin is the only role allowed to create these three manager roles — enforced via the role-creation guard from Phase 0
Receive product request API	View/action Branch → Admin requests	Built on Phase 6's shared request engine — this phase just wires the Admin-side actions (approve/reject/partial-approve/forward to Kitchen)
Add pricing API	Set cost price per product	Admin-only write; Warehouse's inventory reads must omit this field entirely, not just hide it client-side
Receive distribution request API	View/action Warehouse → Admin PO requests	Same shared request engine, Admin-side actions (approve/partial-approve → queue)
Employees (read) API	List all users in the restaurant	Wider scope than other managers — Admin is the one role allowed to see across all locations, per the hierarchy rule
Orders / Branches / Kitchen / Inventory (read) APIs	Aggregate read views across all of Admin's locations	Straightforward scoped reads over Phase 3–5's tables — no new storage, just cross-location query endpoints

Phase 3 — Warehouse Portal Backend

Module / API area	What it covers	Key backend / DB considerations
Warehouse login	Auth via Admin-issued credentials	Reuses Phase 0's single login endpoint — no separate auth path
Create sub-person API	Warehouse Manager adds staff into the shared User table	Sub-users must be invisible to other Warehouse Managers and to Kitchen/Branch — enforced by the created_by_id scoping from Phase 0
Request stock from Admin API	Raise a PO-style request	Writes into Phase 6's shared request table (WAREHOUSE_TO_ADMIN_PO type)
Add stock/ingredients API	Record incoming stock, batch, expiry	Writes a StockMovement record — never a blind quantity overwrite, for audit traceability
Accept release request API	Approve incoming requests from Branch Manager / Kitchen	Warehouse-side action on Phase 6's KITCHEN_TO_WAREHOUSE workflow
Release stock to Kitchen API	Mark a request Dispatched, decrement warehouse inventory	Single status-transition endpoint; triggers the notification Kitchen receives (Phase 6)
Update ingredient quantity API	Manual adjustment with reason	Also writes a StockMovement — same pattern as stock intake
Expiry / waste API	Mark item expired/wasted; near-expiry alert feed	Backs the daily expiry-scan job (Phase 6); Warehouse view must never include costPrice on any response

Phase 4 — Cloud Kitchen Portal Backend

Module / API area	What it covers	Key backend / DB considerations
Kitchen login	Auth via Admin-issued credentials	Reuses Phase 0's login endpoint
Create sub-chef API	Kitchen Manager adds staff into the shared User table	Same created_by_id scoping as Warehouse's sub-person creation
Waste product API	Log wasted/spoiled items with reason code	Same reason-code pattern as Warehouse's wastage log — reuse the enum/table design, don't fork it
Request stock from Warehouse API	Raise a material request	Writes into Phase 6's KITCHEN_TO_WAREHOUSE request type
Product-count notification to Admin	Kitchen reports counts; Admin gets notified	Event-driven via Phase 6's shared notification pipeline, not a polling job
Receive distribution request API	Admin's distribution instruction lands here for approval + dispatch into branches	Kitchen-side actions on Phase 6's BRANCH_TO_ADMIN workflow once Admin has forwarded it
Receive product request from branch API	Branch requests routed to Kitchen after Admin approval	Same shared workflow — Kitchen only sees requests once they reach FORWARDED_TO_KITCHEN
Expiry stickers API	Generate/print expiry label data for a batch	Backend just returns structured label data (product, batch, expiry date) — rendering/printing is a frontend/print-service concern

Phase 5 — Branch Portal Backend

Module / API area	What it covers	Key backend / DB considerations
Branch login	Auth via Admin-issued credentials	Reuses Phase 0's login endpoint
Add salesperson / cashier / order-taker API	Branch Manager adds staff, position-based (RBAC)	Same created_by_id scoping; role-creation guard restricts Branch Manager to these three sub-roles only
Request product from Admin API	Raise a stock/product request	Writes into Phase 6's BRANCH_TO_ADMIN request type — the longest of the four workflows
Order summary API	Take/record customer orders	Feeds both branch inventory deduction and the daily sales sync below
Daily sales sync API	Push branch sales totals to Admin	Should be near-real-time, not an overnight batch — Admin's Analysis (Phase 7) depends on current data
Branch inventory API	Current on-hand quantity per product, this branch only	Credited automatically when a request is marked Received — single source of truth, not manual entry
Waste / expiry API	Log wasted/expired branch stock	Same reason-code and StockMovement pattern as Warehouse/Kitchen
Forecasting/planning data feed	Send branch sales + inventory history upstream	Raw data pipeline into Phase 7 — Branch only produces data here, doesn't compute anything
Planning & forecasting (read) API	Display Admin's planning/forecasting output, branch-scoped	Depends on Phase 7; build the read endpoint shell now, wire real data once Phase 7 lands
Sub-kitchen API	Final prep/shaping tracking within the branch	Modeled as a lightweight kitchen-like inventory/production log scoped to the branch, not a full Kitchen entity
Customer details API	Store customer records against orders	Keep minimal (name/phone) unless the spec later calls for full CRM fields

Phase 6 — Shared Request Engine, Notifications & Audit Log

This phase is what makes “one request engine, one notification system” (used identically by all four workflows) actually true in practice — build it once, reference it everywhere in Phases 1–5.

Module / API area	What it covers	Key backend / DB considerations
Shared request/status model	Single Request table + type field covering all 4 workflows, with a per-type allowed-transitions map	Define the transition rules once (Branch→Admin, Kitchen→Warehouse, Warehouse→Admin PO, Admin→Super Admin plan) — portals must never diverge into separate copies of “what status means what”
Transition/state-machine engine	One PATCH /requests/:id/status endpoint enforcing legal moves per type	Illegal transitions (e.g. Pending → Received directly) must be rejected server-side, not just discouraged in the UI
Partial-approval handling	Per-item approved-quantity support on any request type that allows it	Store quantityApproved per line item, distinct from quantityRequested — needed for Warehouse PO, Branch, and Plan-change flows
Stock-movement side effects	Dispatch/Allocate/Receive transitions trigger inventory quantity changes	Keep this transactional with the status update — a request marked Dispatched with no matching StockMovement is a data-integrity bug
Notification engine	Single table + service, used by every request transition and by expiry/low-stock/billing alerts	Polymorphic entity_type + entity_id so new alert types don't need schema changes
Expiry/low-stock alert job	Scheduled scan across Warehouse/Kitchen/Branch inventory	One job, one table (InventoryItem) — location type is just a field, not three separate scans
Audit log	Append-only log of every mutating action across all portals	Write-only by design — never updated, only inserted into; this is what “proper logs for every action” ultimately means

Phase 7 — Analysis, Forecasting & Planning Backend

Module / API area	What it covers	Key backend / DB considerations
Analytics data pipeline	Aggregate Order + InventoryItem history per branch/product/day	Feeds Admin's chart endpoints — this is a read-aggregation layer, not new source-of-truth storage
Analysis (charts) API	Sales trends, inventory turnover, waste rates	Straightforward aggregation queries; chart rendering happens client-side
Forecasting engine (v1)	Rolling-average/trend calculation per product, per branch	Deliberately simple for MVP — leave room to upgrade to a proper model later, don't over-build now
Planning logic	Propose stock/production quantities from forecast; Admin can accept/override	Forecast is an input, never a directive — the override path is core, not optional
Plan distribution	Confirmed plan becomes visible to Kitchen (production target) and Branch (expected stock)	Reuses Phase 6's notification pipeline to push to both portals

Phase 8 — Billing & Subscription Backend

Module / API area	What it covers	Key backend / DB considerations
Billing cycle engine	Track next_billing_date per restaurant, generate invoices	Scheduled job similar in shape to Phase 6's expiry scan
Billing-due notifications	Alert Admin ahead of the next billing date	Reuses Phase 6's shared notification engine
Plan-change request workflow	Admin → Super Admin plan-change requests	Uses Phase 6's shared request engine (ADMIN_TO_SUPERADMIN_PLAN type) — Pending → Approved/Partially Approved/Rejected → Applied → Confirmed
Invoice sharing	Expose invoice history to Admin (read-only)	Already covered by Phase 1's billing-data API; this phase just adds the scheduled generation side

Phase 9 — Additional Recommended Backend Work

Module / API area	What it covers	Key backend / DB considerations
Barcode/QR handling	Accept scanned codes at Warehouse/Kitchen/Branch stock intake instead of manual entry	Backend just needs a lookup endpoint for a scanned code — the scan UI itself is frontend
Expiry sticker generation	Structured label data (product, batch, expiry) for printing	Already scaffolded in Phase 4; formalize as a shared endpoint reusable by Warehouse and Kitchen alike
Role-based notification preferences	Let users opt in/out of specific alert types	Small preferences table per user; filter Phase 6's notification pipeline against it
Multi-branch comparison queries	Backend queries powering cross-branch comparison views for Admin	Aggregation over Phase 7's analytics pipeline — no new storage, just new query endpoints

Phase 10 — Testing, Integration & Deployment

Module / API area	What it covers	Key backend / DB considerations
Full backend/DB regression pass	End-to-end flow across all 5 portals, including edge cases (rejections, partial approvals, expired stock)	Test the hierarchy-visibility rule specifically — a sub-user created by one manager must never leak to another
DB indexing & optimization	Index tuning on high-read tables (InventoryItem, Request, Notification, AuditLog)	Do this once real data volume patterns are visible from earlier phases, not blindly upfront
UAT bug fixes (backend/DB)	Fixes from client UAT feedback	Keep a prioritized list — backend fixes often block frontend fixes, sequence accordingly
Production deployment & DB migration	Environment config, Alembic migration scripts, monitoring setup	Test migrations against a staging copy of real-shaped data before running on production